

Panduan dosen Lab 01 - Cloud Model Lab, Git, dan API pertama

Durasi demo: 90 menit. **Capaian:** mahasiswa membedakan model layanan dan deployment cloud, menjelaskan tanggung jawab pengguna/penyedia, menjalankan halaman serta dua fungsi API, lalu menulis keputusan arsitektur per komponen. Gunakan [modul mahasiswa](#) sebagai lembar praktik dan [panduan Git](#) untuk pengumpulan. Repo materi mandiri adalah [SeedFlora/meet1CloudService](#).

Sebelum kelas

1. Pastikan Node.js 20+ dan Git tersedia. Dari **root repo Lab 01**, jalankan `cd starter, node --version, npm test`. Tidak ada dependency eksternal sehingga `npm install` tidak diperlukan. Pada uji ulang lokal 7 Oktober 2026, **18 tes lulus dan 0 gagal**.
2. Cek port 3000 kosong. Mulai server dengan `npm run dev`; buka `http://localhost:3000`. Hentikan dengan Ctrl+C di akhir kelas.
3. Siapkan dua terminal: satu untuk server, satu untuk `curl.exe` di PowerShell atau `curl` di Bash. Jangan proyeksikan token, password, atau data mahasiswa.
4. Jika GitHub/Vercel belum dapat diakses, semua kode, API, dan analisis kasus tetap dapat dikerjakan lokal. Deployment adalah perluasan yang membutuhkan akun dan izin repo.

Alur 90 menit

Menit	Aksi dosen	Aksi mahasiswa	Bukti/checkpoint
0-10	Terangkan lima karakteristik NIST dan shared responsibility.	Sebutkan contoh IaaS/PaaS/SaaS/BaaS/FaaS.	Satu contoh layanan dan hal yang tetap dikelola pengguna.
10-20	Tunjukkan struktur <code>starter/</code> , <code>student.json</code> , dan <code>npm test</code> .	Isi tiga field identitas publik yang sesuai, lalu uji.	<code>pass 18, fail 0</code> ; data contoh sudah diganti.
20-35	Jalankan <code>npm run dev</code> ; pilih IaaS, PaaS, SaaS.	Catat lapisan yang pindah pengelolanya.	Tabel tanggung jawab di laporan.
35-50	Uji GET <code>/api/hello</code> , <code>/api/recommend</code> , dan faktor tidak dikenal.	Ulangi dengan <code>curl.exe/curl</code> ; baca status HTTP.	200, 200, 400.
50-70	Pilih satu kasus A-D dan contohkan analisis per komponen.	Isi worksheet dengan tiga alasan, trade-off, alternatif.	Keputusan yang dapat dijelaskan, bukan hanya output engine.
70-82	Demonstrasikan issue, commit, diff staged, push.	Simpan <code>hasil/lab01.md</code> dan bukti sendiri.	Repo pribadi dan commit; Actions menjalankan 18 tes.
82-90	Bahas lima pertanyaan, error umum, dan penutup.	Serahkan tautan repo/issue; Vercel bila ditugaskan.	Jawaban singkat dan status tes.

Demo terminal dan cara kerjanya

Dari root repo di **PowerShell**:

```
Set-Location .\starter
node --version
npm test
npm run dev
```

`npm test` memakai runner bawaan Node untuk logika engine, endpoint, dan struktur data. `npm run dev` menjalankan server lokal pada 3000. Pada terminal kedua gunakan:

```
curl.exe -i http://localhost:3000/api/hello
curl.exe -i 'http://localhost:3000/api/recommend?spiky=1&smallTeam=1'
curl.exe -i 'http://localhost:3000/api/recommend?factors=ngawur'
```

Status 200 pertama membuktikan fungsi `hello` menjawab. Status 200 kedua menunjukkan engine menerima faktor `spiky` dan `smallTeam`; baca alasan dan skor sebelum menyimpulkan. Status 400 ketiga menunjukkan validasi input berjalan. Di Bash gunakan `curl` tanpa `.exe`. Jangan menyamakan tes hijau dengan identitas sudah benar: `student.json` contoh masih memenuhi syarat format tes.

Screenshot untuk ditunjukkan bersama command

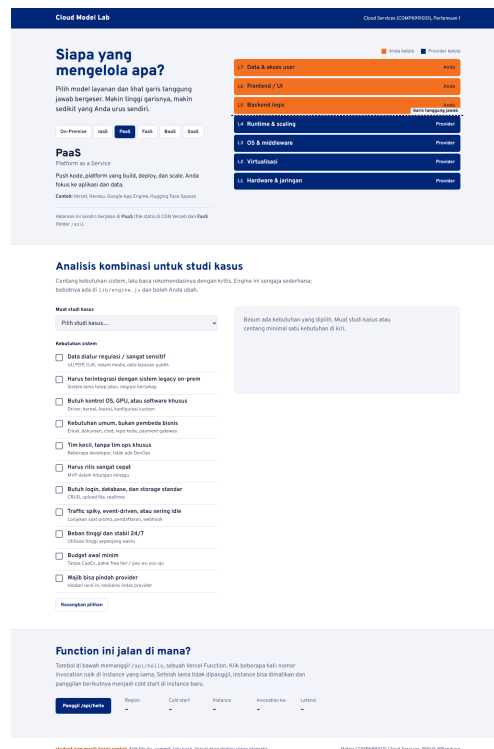
```
Lab 01 - uji lokal aktual
npm test (Node.js)
tests 18
pass 18
fail 0
duration_ms 136.0422

HTTP request ke server lokal
/api/hello -> HTTP 200
/api/recommend?spiky=1&smallTeam=1 -> HTTP 200
/api/recommend?factors=ngawur -> HTTP 400

Transkrip uji aktual ditata ulang agar terbaca; lihat bukti pribadi Anda untuk penilaian.
```

Run ulang Node dan request lokal Lab 01

Perintah: `npm test` dan tiga GET di atas. **Fungsi:** mengunci baseline kode serta kontrak HTTP. **Cara kerja:** Node menguji fungsi, kemudian server menerima request dan mengembalikan status. **Baca:** 18/18 tes, 200/200/400. Cuplikan terminal adalah keluaran aktual yang ditata agar terbaca.



Tampilan Cloud Model Lab terbaru dari browser lokal

Perintah: `npm run dev`, buka `http://localhost:3000`. **Fungsi:** memperlihatkan model layanan dan faktor kasus.

Cara kerja: browser mengambil HTML/JS dari server yang sama dengan API. **Baca:** panel model, daftar faktor, serta area rekomendasi; identitas contoh perlu diganti oleh mahasiswa.

```
PowerShell - labs/01/starter

npm test
✓ GET /api/hello: cold start hanya di panggilan pertama instance (24.4084ms)
✓ GET /api/recommend tanpa parameter mengembalikan petunjuk pemakaian (0.6105ms)
✓ GET /api/recommend dengan query string (0.9874ms)
✓ POST /api/recommend dengan body JSON (0.7829ms)
✓ POST dengan JSON rusak -> 400 (0.4687ms)
✓ POST tanpa factor valid -> 400 (0.3805ms)
✓ input kosong ditolak dengan pesan yang jelas (0.9422ms)
✓ normalizeFactors menerima array, objek, dan query string (0.905ms)
✓ W1 marketplace UMKM: public + BaaS/PaaS/FaaS (0.5483ms)
✓ W1 komponen pembayaran/email: cukup SaaS (0.196ms)
✓ W2 bank: analisis per komponen menghasilkan model berbeda (0.2974ms)
✓ data teregulasi selalu memunculkan catatan region Indonesia (0.1679ms)
✓ wajib bebas lock-in mendorong multi-cloud (0.1247ms)
✓ kombinasi selalu berisi tepat satu model utama (0.9626ms)
✓ stack: tanggung jawab Anda tidak pernah naik dari kiri ke kanan (0.2661ms)
✓ semua bobot hanya memakai factor yang terdaftar (0.2299ms)
✓ semua preset di cases.json memakai factor yang valid (0.1642ms)
✓ student.json valid dan field wajib terisi (1.7229ms)
  tests 18
  suites 0
  pass 18
  fail 0
  cancelled 0
  skipped 0
  todo 0
  duration_ms 111.9897
```

Tangkapan layar tes 18 dari paket awal

Perintah: `npm test` di `starter/`. **Fungsi:** menunjukkan bentuk ringkasan runner Node. **Cara kerja:** `node --test` menemukan tiga file tes. **Baca:** `pass 18` dan `fail 0`; gunakan bukti run terbaru di atas untuk kelas ini.

Siapa yang mengelola apa?

Pilih model layanan dan lihat garis tanggung jawab bergeser. Makin tinggi garisnya, makin sedikit yang Anda urus sendiri.

On-Premise **IaaS** PaaS FaaS BaaS SaaS

PaaS

Platform as a Service

Push kode, platform yang build, deploy, dan scale. Anda fokus ke aplikasi dan data.

Contoh: Heroku, Vercel, Google App Engine, PaaSless, Heroku, PaaSless

Halaman ini sendiri berjalan di PaaS (file statis di CDN Vercel) dan PaaS (folder /api).

Anda kelola **Provider kelola**

- L7 Data & akses user **Anda**
- L6 Frontend / UI **Anda**
- L5 Backend logic **Anda tanggung jawab**
- L4 Runtime & scaling **Provider**
- L3 OS & middleware **Provider**
- L2 Virtualisasi **Provider**
- L1 Hardware & jaringan **Provider**

Analisis kombinasi untuk studi kasus

Cerang kebutuhan sistem, lalu baca rekomendasi dengan kritis. Enggak ini sebagai acuan, bobotnya ada di L7-L8 yang ada.

Mau studi kasus...

Pilih studi kasus...

Kelebihan sistem

- ☐ Data diatur register / sangat sensitif
- ☐ Data diatur, akses mudah, dan terkontrol
- ☐ Harus terintegrasi dengan sistem legacy on-prem
- ☐ Butuh kontrol OS, GPU, atau software khusus
- ☐ Butuh kontrol, bukan pembaruan biasa
- ☐ Tim kecil, tanpa tim ops khusus
- ☐ Harus bisa sangat cepat
- ☐ Butuh login, otentikasi, dan storage standar
- ☐ Traffic spike, event-driven, atau scaling
- ☐ Beban tinggi dan stabil 24/7
- ☐ Budget awal minim
- ☐ Mampu bisa pindah provider

Keuntungan pilihan

Function ini jalan di mana?

Tombol di bawah memanggil /api/me/1, sebuah Vercel Function. Kita beberapa kali nomor invocation baik di instance yang sama. Setelah lama tidak dipanggil, instance bisa dimatikan dan panggilan berikutnya menjadi cold start di instance baru.

Region Cold start Instance Invocation bar Latency

Anda kelola **Provider kelola**

Halaman awal aplikasi

Perintah: `npm run dev`, lalu buka URL lokal. **Fungsi:** orientasi UI. **Cara kerja:** JavaScript memuat `student.json`, diagram tanggung jawab, dan kasus. **Baca:** setiap panel sebelum mahasiswa mengubah faktor.

Siapa yang mengelola apa?

Pilih model layanan dan lihat garis tanggung jawab bergeser. Makin tinggi garisnya, makin sedikit yang Anda urus sendiri.

On-Premise IaaS PaaS FaaS BaaS **SaaS**

SaaS

Software as a Service

Aplikasi jadi yang tinggal dipakai. Data dan hak akses user tetap tanggung jawab Anda.

Contoh: GitHub, Google Workspace, Microsoft 365

Halaman ini sendiri berjalan di PaaS (file statis di CDN Vercel) dan FaaS (folder /api).

Anda kelola **Provider kelola**

- L7 Data & akses user **Anda, tetap! Garis tanggung jawab**
- L6 Frontend / UI **Provider**
- L5 Backend logic **Provider**
- L4 Runtime & scaling **Provider**
- L3 OS & middleware **Provider**
- L2 Virtualisasi **Provider**
- L1 Hardware & jaringan **Provider**

SaaS pada diagram tanggung jawab

Perintah: klik pilihan **SaaS**. **Fungsi:** diskusi batas pengelolaan. **Cara kerja:** UI mengganti garis dan warna tiap lapisan. **Baca:** data dan akses tetap perlu dikelola pengguna, sedangkan operasi software/platform ada pada penyedia.

Lab 01: cuplikan respons API nyata — keluaran aktual dari salinan uji

```
$ GET /api/recommend?spiky=1&smallTeam=1
{
  "ok": true,
  "factors": [
    "smallTeam",
    "spiky"
  ],
  "deployment": {
    "id": "public",
    "name": "Public cloud",
    "definition": "Infrastruktur milik provider, dipakai bersama banyak tenant lewat internet.",
    "score": 3,
    "reasons": [
      {
        "factor": "spiky",
        "label": "Traffic spiky, event-driven, atau sering idle",
        "weight": 2
      },
      {
        "factor": "smallTeam",
        "label": "Tim kecil, tanpa tim ops khusus",
        "weight": 1
      }
    ]
  },
  "ranking": [
    {
      "id": "public",
      "name": "Public cloud",
      "score": 3
    },
    {
      "id": "hybrid",
      "name": "Hybrid cloud",
      "score": -1
    }
  ],
  ... (respons JSON dipotong untuk screenshot)
```

Respons rekomendasi dari API

Perintah: `GET /api/recommend?spiky=1&smallTeam=1`. **Fungsi:** hubungkan form browser dengan JSON API. **Cara kerja:** handler memvalidasi faktor lalu memanggil engine skor. **Baca:** status, faktor yang diterima, kombinasi model, alasan, dan trade-off.

Analisis kombinasi untuk studi kasus

Centang kebutuhan sistem, lalu baca rekomendasinya dengan kritis. Engine ini sengaja sederhana; bobotnya ada di `lib/engine.js` dan boleh Anda ubah.

Must studi kasus

Aplikasi web + backend

Startup 3 developer tanpa tim ops. MVP harus live dalam 6 minggu: login pembeli dan penjual, katalog, upload foto produk. Traffic bisa naik 10x saat promo tanggal kembar.

Kebutuhan sistem

☐ Data diatur regulasi / sangat sensitif

UU PDP, OJK, rekam medis, data layanan publik

☐ Harus terintegrasi dengan sistem legacy on-prem

Sistem lama tetap jalan, migrasi bertahap

☐ Butuh kontrol OS, GPU, atau software khusus

Driver, kernel, lisensi, konfigurasi custom

☐ Kebutuhan umum, bukan pembeda bisnis

Email, dokumen, chat, repo kode, payment gateway

☒ Tim kecil, tanpa tim ops khusus

Beberapa developer, tidak ada DevOps

☒ Harus rilis sangat cepat

MVP dalam hitungan minggu

☒ Butuh login, database, dan storage standar

CRUD, upload file, realtime

☒ Traffic spiky, event-driven, atau sering idle

Lonjakan saat promo, pendaftaran, webhook

☐ Beban tinggi dan stabil 24/7

Utilisasi tinggi sepanjang waktu

☒ Budget awal minim

Tanpa CapEx, pakai free tier / pay-as-you-go

☐ Wajib bisa pindah provider

Hindari lock-in, resiliensi lintas provider

Kosongkan pilihan

Deployment model

Public cloud

Infrastruktur milik provider, dipakai bersama banyak tenant lewat internet.

Public cloud

Hybrid cloud

Multi-cloud

Private cloud

7

-2

-4

-6

Kombinasi service model

BaaS

Backend as a Service

utama

skor 7

+3 Butuh login, database, dan storage standar

+2 Harus rilis sangat cepat

+1 Tim kecil, tanpa tim ops khusus

+1 Budget awal minim

Contoh layanan: Supabase, Firebase, AWS Amplify

PaaS

Platform as a Service

pendukung

skor 6

+2 Tim kecil, tanpa tim ops khusus

+2 Harus rilis sangat cepat

+1 Traffic spiky, event-driven, atau sering idle

+1 Budget awal minim

Contoh layanan: Vercel, Heroku, Google App Engine, HF Spaces

FaaS

Function as a Service

pendukung

skor 5

+3 Traffic spiky, event-driven, atau sering idle

+1 Tim kecil, tanpa tim ops khusus

+1 Budget awal minim

Contoh layanan: Vercel Functions, AWS Lambda, Cloud Run Functions

Catatan dan trade-off

- BaaS/PaaS mempercepat rilis, tapi mengikat Anda ke API provider dan biayanya bisa melonjak saat skala besar. Siapkan exit plan (mis. Supabase = PostgreSQL standar) dan pasang budget alert.

Dihitung oleh FaaS /api/recommend dalam 4 ms.

curl "http://localhost:3000/api/recommend?smallTeam=1&fastLaunch=1&noDk"

Salin perintah curl

Preset kasus pada browser

Perintah: pilih satu preset di UI. **Fungsi:** mulai diskusi keputusan per komponen. **Cara kerja:** preset mencentang faktor yang kemudian dihitung engine. **Baca:** output adalah hipotesis awal; minta mahasiswa menyebutkan faktor yang belum terwakili.

Kunci diskusi dan semua kasus

Kasus	Contoh keputusan	Alasan dan keberatan yang perlu muncul
A, event kampus	SaaS form jika sederhana; bila perlu kuota/pembayaran, PaaS frontend + BaaS data + FaaS webhook pada public cloud.	Lonjakan singkat, panitia kecil, biaya idle. Uji kuota dan ekspor data; jangan mengumpulkan data yang tidak diperlukan.
B, klinik	Pertimbangkan SaaS RME dengan kontrak, lokasi data, audit akses, dan ekspor; integrasi khusus dapat dipisah.	Data sensitif dan dua staf IT membuat operasi private penuh berat. Verifikasi aturan serta kewajiban terbaru sebelum keputusan nyata.
C, AI dubbing	Training GPU batch pada IaaS/container; inference pada layanan terpisah; SaaS untuk kebutuhan umum.	GPU periodik, klien global, keinginan pindah penyedia. Spot dapat berhenti; checkpoint model dan biaya transfer perlu diuji.
D, PPDB	Integrasi data internal dekat sistem pemda; frontend/antrian elastis pada layanan yang memenuhi ketentuan.	Trafik musiman, data warga, koneksi ke sistem lama. Uji beban, backup/restore, serta aturan lokasi/akses yang berlaku.

Lima jawaban inti: (1) pada PaaS/FaaS tim tetap memegang kode/fungsi, data, konfigurasi dan akses; (2) public cloud adalah model deployment, multi-cloud adalah arsitektur lintas penyedia untuk workload; (3) *spiky* dan *smallTeam* memberi alasan kapasitas elastis dan sedikit operasi, tetapi aturan lokasi/biaya dapat mengubah pilihan; (4) HTTP 400 berasal dari validasi faktor yang tidak dikenal; (5) visibilitas repo Git dan situs deploy adalah pengaturan berbeda. Minta satu alternatif yang ditolak pada setiap worksheet. Rubrik di [starter/exercises/rubric.md](#) menilai alasan dan trade-off, bukan mencocokkan model engine secara buta.

Git, penilaian, dan troubleshooting

Dari root repo, salin `hasil/TEMPLATE_LAPORAN.md` menjadi `hasil/lab01.md`. Jalankan `git status --short`, `git add starter hasil/lab01.md hasil/bukti`, `git diff --cached --name-only`, `git diff --cached --check`, baru `git commit` dan `git push`. Workflow root `.github/workflows/verify.yml` menjalankan `npm test` pada push. Untuk Vercel opsional, impor repo pribadi dan set **Root Directory** `starter`, lalu cek halaman dan dua endpoint HTTPS. Jangan tampilkan token pada proyektor.

Gejala	Diagnosis/aksi
<code>npm</code> tidak ditemukan	Periksa instalasi Node dan terminal yang digunakan.
Port 3000 terpakai	Hentikan server lama; jangan ubah port tanpa menyesuaikan URL bukti.
404 endpoint	Jalankan server dari <code>starter/</code> , bukan root repo.
18 tes lulus tetapi nama contoh tampil	Isi <code>starter/student.json</code> dan refresh browser; tes hanya memeriksa struktur.
Vercel halaman ada tetapi API gagal	Periksa Root Directory <code>starter</code> , log deployment, dan commit yang terdeploy.

Batas uji yang dilakukan: kode Node, halaman browser lokal, dan status API 200/200/400 diuji ulang pada komputer ini. Push repo dan deployment Vercel ditangani saat publikasi/kelas dan perlu akun masing-masing.